

面试押题 09 | Ray、FlashInfer、Mirage：接口兼容与正确性

对应简历「其他项目（3 个 PR 已合入）」。核验日期：2026-09-29；作者账号 XFDG。本文依据最终 diff 和既有验证记录，未在本轮重跑测试；不把测试兼容性修复说成算子加速，也不把 demo workaround 说成一般性数学保证。

1. 三个仓库分别做什么

仓库	功能与我修改的位置	PR / 合入日期（UTC）
Ray	分布式运行时与 AI 库；本次修改 Serve 的 gRPC server handler 注册适配	#66502 / 2026-09-28
FlashInfer	LLM 推理 GPU kernel 库；本次修改 BF16 rank-major session 的 CPU 契约测试加载方式	#5171 / 2026-09-15
Mirage	将 LLM 编译为 persistent/MegaKernel 的项目；本次修改 Qwen3 demo 的 lm_head padding 初始化	#755 / 2026-08-26

简单技术原理与典型应用

- **Ray**：以 task、actor 和对象传递组织分布式执行，上层提供 Serve 等 AI 库；Serve 将模型部署为可扩展的服务并接收 HTTP/gRPC 请求。常用于模型在线服务和分布式 AI 工作流。我的改动在 gRPC 接入适配，不是集群调度算法。[官方介绍](#)
- **FlashInfer**：为推理框架提供 attention、采样、GEMM 等 GPU 内核，通过专门的数据布局与调度减少计算和访存开销。常用于 LLM serving 的 prefill/decode；本次 PR 仅保证 CPU 契约测试在源码和 wheel 环境都能加载，不宣称提升推理性能。[官方介绍](#)
- **Mirage**：把模型计算编译、组织为 GPU 上持续执行的任务图/MegaKernel，减少逐算子由 CPU 发起的调度与启动开销。典型应用是低延迟 LLM 推理；图的物理对齐尺寸仍需与模型逻辑尺寸区分。本次工作是 Qwen3 demo 的权重 padding 修复，不是开发整个编译器。[官方介绍](#)

2. 45 秒 STAR 口述

S：AI Infra 的问题不都出在算术 kernel，还会出在依赖升级、源码/安装包布局差异和图对齐边界。**T**：我针对上游 issue 追踪到具体适配层并提交可审查修复。**A**：在 Ray 补齐 registered handler 的代理重写，在 FlashInfer 用 distribution metadata 保持 CPU 测试的可选驱动导入契约，在 Mirage 修正 Qwen3 demo 的 padding 配置。**R**：三个 PR 均已合入；其中前两项有明确的定向测试结果，Mirage 的改动范围和理论限制单独说明，不用「已合入」代替完整正确性证明。

3. Ray #66502：依赖多了一条注册通路，代理也要覆盖

3.1 背景和问题

Ray Serve 的 gRPC 请求需要经过 proxy 才能正确路由到部署。原适配覆盖 `add_generic_rpc_handlers()`，但 grpcio 还存在 `add_registered_method_handlers()` 通路；只改 generic handler 不足以保证另一条注册路径仍通过 Serve。

我的责任范围是 `python/ray/serve/_private/grpc_util.py` 与对应 unit test, 不是重写 Ray 的 actor 调度或 Serve 控制面。PR 正文及最终 diff

3.2 我的思路和解决方法

1. 将已有 handler 重写抽成 `_override_method_handler()`, generic 和 registered 两条路径共享, 避免序列化和 streaming 类型处理逐渐分叉。
2. registered path 把 service/method 组成 `/{{service_name}}/{{method_name}}`, 将四类 RPC callable 接到 `service_handler_factory`。
3. 保留 passthrough service, 不应代理的服务原样交给 grpcio。
4. 用 `getattr(super(), "add_registered_method_handlers", None)` 兼容没有这个 API 的旧 grpcio, 沿用 generic 注册作为兼容通路。

3.3 结果和测试

- 2026-09-28 21:04 UTC 合入; 旧 #66494 被此 PR 替代, 不重复计数。
- grpcio 1.84.0 下定向测试 **5 passed, 11 deselected**; Ruff、format、diff 检查通过。
- 新增测试覆盖 registered handler 被重写、四类 streaming callable、passthrough 原样保留、缺失注册 API 的 fallback。
- 测试使用 `--noconftest`, 因为源码 conftest 依赖的 API 与安装 Ray wheel 不一致; 因此这不是整个 Ray 测试套件通过的声明, 也没有吞吐提升数据。验证来源

一句话口述: 我补齐 grpcio 新注册路径在 Serve proxy 中的适配, 并保持旧版本和 passthrough 服务兼容。

4. FlashInfer #5171: CPU 契约测试不能依赖完整源码树或隐式 CUDA 导入

4.1 背景和问题

Nightly package-test 隔离环境只复制 `tests/` 与 `pytest.ini`。原测试根据测试文件位置寻找源码树中的 `session.py`, 安装包环境不存在该相对路径, 导致测试在真正验证 session 契约前就失败。

同时, 该测试要求不导入整个 FlashInfer 包树, 以保持无 CUDA driver 的 CPU 契约测试可运行。简单改成普通 package import 或 `importlib.resources.files()` 会引入父包导入副作用。最终 diff

4.2 我的思路和解决方法

1. 源码树路径存在时保持原 fast path。
2. 否则通过 `importlib.metadata.distribution("flashinfer-python")` 和 `Distribution.locate_file()` 定位已安装文件, 不导入 FlashInfer 包树。
3. 用明确文件路径加载 session module; 打包源码 manifest 断言相对实际 module 路径计算, 不能继续相对原测试目录。
4. `pyproject.toml` 是源码树资产, 仅当隔离安装环境没有它时跳过该断言, 其他 package/source contract 检查继续执行。

4.3 结果和测试

- 2026-09-15 合入, 只修改 `tests/moe_ep/test_bf16_rank_major_cuda_session_cpu.py`。
- 源码树整文件 **40 passed**; 模拟 Nightly 安装包隔离布局 **39 passed, 1 skipped**, skip 精确对应源码专属 pyproject 断言。
- `py_compile` 与 `git diff --check` 通过; 没有本地 pre-commit 环境, 格式检查不伪报为本地执行。验证来源

贡献边界：这是 BF16 rank-major session 的测试/安装包兼容性修复，不是 BF16 MoE EP kernel 开发，更不是九坤 RL FlashInfer CUDA Graph hang 的上游原创修复。RL 事件是另一条回移上游修复与集成验证工作线。

一句话口述：我让同一套 CPU 契约测试在源码与已安装 wheel 两种布局下工作，同时守住不隐式加载 CUDA driver 的边界。

5. Mirage #755：区分 padding 工作区与真实词表

5.1 背景和问题

Qwen3 demo 为任务图整除约束把 lm_head 扩到 153600 行。原 padding 权重行为零，因此对应 dot product logit 为零；argmax 扫描了 padded width，如果真实 logits 全为负，padding 索引可能被选中，得到词表之外的 token。

这是「物理计算尺寸大于逻辑有效尺寸」的边界问题：对齐方便了调度，但输出选择仍应遵守真实

vocab_size。PR 与最终 diff

5.2 实际合入了什么

修改 9 个 Qwen3 demo，在扩展 lm_head 权重时把 padding 填充值从 0 改为 -1e4。没有修改 argmax_partial / argmax_reduce CUDA kernel，也没有在 argmax 接口中增加 vocab bound。2026-08-26 合入。

5.3 必须讲清的理论边界

不能说「把 padding logit 设为 -1e4，所以所有输入都不可能越界」。实际改的是权重：对 hidden 向量 h，padding 行的结果是 $-1e4 * \text{sum}(h)$ ，其符号依赖 h；它不等价于在最终 logits 上显式 mask。这里的等式是对最终 diff 的直接数学分析，不是重新跑出的性能或模型结果。

更严格的后续方案应在输出选择处使用真实词表边界，例如只归约 $\text{index} < \text{vocab_size}$ ，或对 padding logits 显式 mask 为 -inf 后再 argmax。这个方案可在面试中作为复盘改进，但不能说本 PR 已经实现。

5.4 结果与验证口径

- 可核验事实：9 个 demo 的 padding 初始化调整已合入，修复方向针对异常 token 症状。
- PR 正文记载 Qwen3 BF16 的复现与修后表现，但未给出可独立复核的完整运行日志；且同时声明未完成 GPU CI，因此不能把它包装成覆盖任意 hidden/shape 的证明。
- 原 tests/runtime_python/test_argmax.py 注入一个已知最大值，本就不敏感于全负真实 logits 和 padding 竞争，不能拿「原测试不变」证明新边界被回归覆盖。
- 如果继续开发，应新增全负真实 logits、不同 hidden 和词表边界样例，并显式验证 token 范围；本轮仅做面试资料整理，不擅自修改上游实现。

稳妥口述：我向 Mirage 的 Qwen3 demo 提交并合入 padding 配置修复，处理异常 token 问题；复盘来看，若要提供一般性保证，应该在 logits/argmax 层显式限制真实词表范围。

6. 高频追问与具体答法

Q1. Ray 修了几行适配，为什么能影响服务行为？

proxy 必须拦住所有实际使用的 handler 注册入口。依赖增加或启用另一条入口后，即使原入口仍正确，也可能绕过代理；因此排查要看调用路径，而不是只看旧函数有没有被改坏。

Q2. 为什么 Ray 要同时处理四类 RPC?

gRPC 支持 unary-unary、unary-stream、stream-unary、stream-stream。共享 helper 将每类连接到对应 factory 类型，避免只修普通请求而漏掉流式服务；测试验证生成的 callable 与完整服务路径。

Q3. 为什么把 response_serializer 置空?

这是延续原 Serve 代理路径的约定：由 Serve 的处理链负责返回序列化数据，不能再套用原始 protobuf handler 的序列化器。重构把同一行为用于两个入口，而不是另造一套协议。

Q4. passthrough 为什么不能也统一重写?

它的语义就是交给原服务实现处理；统一代理会改变本来不属于 Serve 应用路由的服务。测试不仅看调用成功，还断言原 method handler 容器被原样传给基类。

Q5. FlashInfer 为什么不用相对 tests 的路径?

测试代码与包安装位置没有稳定的相对关系，Nightly 刻意拆开它们。distribution metadata 才是已安装分发包资源位置的证据；源码路径只能作为存在时的快路径。

Q6. importlib.resources 不也是标准库吗，为什么不能用?

问题不在是否标准库，而在该调用会导入父包。这里父包初始化可能拉起 CUDA driver 依赖，违反 CPU 契约；metadata 查分发包位置不执行 package import，副作用边界更窄。

Q7. 39 passed、1 skipped 是不是把失败测试藏起来?

需要解释 skip 的前提与内容：仅缺失源码树的 pyproject 时跳过源码声明检查；源码树仍执行该项，其余 manifest、source/hash 和 session contract 继续验证。不能扩大成安装环境全部测试 skip。

Q8. Mirage 为什么 padding=0 会出错?

零权重乘 hidden 得到零。如果真实 logits 都小于零，扫描整个扩展词表的 argmax 会选 padding。根本问题是对齐后的物理范围泄漏进语义选择范围。

Q9. 权重 padding=-1e4 一定对应负 logit 吗?

不一定。它等于 $-1e4 * \text{sum}(\text{hidden})$ ；hidden 的和可能为负或零。因此严格方案应在最终 logits 或归约索引上 mask，本次已合入代码只是 demo 层方案，不能夸大成数学完备修复。

Q10. 为什么没有写这三个 PR 的加速百分比?

它们的主要验收目标是路由正确、测试可运行、异常输出边界，不是性能优化。结果应该报合入状态与对应回归，不编造性能数字，也不把九坤 RL 性能工作混进 FlashInfer 测试 PR。

Q11. 面试官问哪个 PR 最能证明你的测试能力?

可以展开 FlashInfer：同一套测试分别在 source tree 和安装包隔离布局执行，明确导入副作用和条件 skip；或 Ray：同时检查新入口、旧版本缺 API 和 passthrough，不只验证 happy path。

Q12. 如果现在重新做 Mirage 的修复，你会怎么验?

在 argmax/采样边界定义真实词表范围，构造真实 logits 全负、padding 更大、尾块非整除等反例，断言 $0 \leq \text{token} < \text{vocab_size}$ ；同时测试有效词表内结果与参考一致。这里是后续方案，不声称已经提交或执行。

7. 来源与计数边界

- 3 个 PR 均核对作者与 `merged_at`；日期截至 2026-09-29，不与替代 PR、开放 PR 或内部仓改动重复计数。
- 开发记录：B200 OSS 修复记录、2026-09 开源 issue 跟进。历史 OPEN 是当时快照，当前状态以 PR 为准。
- Mirage PR 正文中关于 BF16 数值范围及「始终不会选中 padding」的说法不作为可靠论证；本稿明确按实际修改位置与数学约束解释。